enewcommandheadrulewidth0.4pt enewcommand

ootrulewidth0.4pt

~~. We present SciTeX LaTeX-based . The system .~~

Summary

Scientific workflows require coordination across manuscript preparation, computational analysis, figure generation, data management, and literature curation, yet these activities remain distributed across disconnected tools, making reproducibility difficult to sustain. SciTeX is an AI-native, modular research platform that unifies these tasks through four core components—Writer, Scholar, Viz, and Code—with optional extensions for statistical analysis. Each module provides standalone functionality while participating in a shared project structure built on lightweight symlinks and global identifiers, enabling provenance-preserving links between code, data, figures, and manuscript text. Containerized builds ensure byte-consistent manuscript output across systems, and standardized metadata captures computational and visual parameters for downstream verification or regeneration. By embedding reproducibility infrastructure directly into routine authoring, visualization, and analysis workflows, SciTeX lowers the activation energy required for transparent scientific practice while maintaining compatibility with existing toolchains. This manuscript, prepared and compiled entirely within SciTeX, demonstrates its end-to-end integration.

Highlights

- SciTeX unifies manuscript writing, computation, figure generation, and literature management in a single reproducible research environment.
- Modular design allows each component to function independently while enabling seamless integration through lightweight symlinks and global identifiers.

- Containerized LaTeX builds and standardized metadata formats ensure consistent manuscripts and reconstructible figures across systems.
- Provenance-preserving workflows link code, data, figures, and text, reducing cognitive load and supporting transparent scientific practice.

- AI-native architecture exposes structured interfaces for automated analysis, figure refinement, and manuscript assistance.

SciTeX: A unified research OS for reproducible scientific workflows

Yusuke Watanabe[a,*]

[a] *SciTeX.ai, Tokyo, Japan*

Keywords: reproducible research , scientific workflow automation , data science platform , AI-native tools , figure provenance , FAIR principles

ˇ 8 figures, 3 tables, 146 words for abstract, and 3581 words for main text

1. Introduction

~~The preparation of scientific manuscripts involves numerous technical challenges that extend beyond the intellectual task of communicating researchfindings. Researchers must navigate complex typesetting systems, manage multiple document versions, coordinate figures and tables~~ Scientific workflows increasingly span heterogeneous environments, including personal laptops, cloud servers, containerized pipelines, and HPC systems. Yet the core activities of research—manuscript writing, computational analysis, figure generation, and literature management—remain distributed across disconnected tools. This fragmentation increases cognitive load, introduces opportunities for error, and makes reproducibility difficult to sustain [1, 2]. Researchers must manually coordinate figures across formats, ~~and ensure reproducible compilation environments~~maintain consistent bibliographies, track versions of manuscripts and data, and ensure that computational analyses remain synchronized with the text that describes them [3, 4]. These burdens often overshadow the substantive scientific work.

Traditional ~~approaches to manuscript preparation typically rely on local LaTeX installations , where specific versions of packages and compilation~~

*Corresponding author. Email: ywatanabe@scitex.ai

~~tools can vary significantly across~~ LaTeX-based manuscript preparation further amplifies these challenges. Local installations vary across machines and over time~~. This variability creates reproducibility challenges, particularly in collaborative environments where multiple authors work on different systems, leading to~~ , producing inconsistent builds and complicating collaboration [5]. Figure generation typically occurs in isolation from manuscript writing, with libraries and scripts lacking standardized metadata or provenance tracking. Literature management tools such as Zotero and Mendeley help organize references but operate independently of manuscript preparation and computational workflows, creating citation drift when documents evolve. Computational analyses executed in Jupyter notebooks or ad hoc scripts provide convenience but lack the structured reproducibility required for long-term verification [6]. As a result, researchers must orchestrate a complex toolchain whose components have no shared model of provenance, reproducibility, or workflow state.

Existing solutions ~~have addressed aspects of this problem. Overleaf and similar cloud-based platforms provide consistent compilation environments but require continuous internet connectivity and .~~

~~The fundamental challenge lies in accommodate diverse content types, multiple output documents, and varying journal requirements while a single source of truth for shared elements. Additionally,~~ address individual components of this ecosystem but stop short of providing a unified workflow. Overleaf ensures consistent LaTeX compilation but requires separate figure generation and data management workflows. Visualization tools such as SigmaPlot and Origin offer polished interfaces but remain disconnected from manuscript and code environments; figures created in these tools become static images that cannot be regenerated or audited. Manubot demonstrates the value of Git-based open writing [7], while Binder and related technologies provide reproducible computational environments [8]. Still, no existing platform integrates writing, computation, visualization, and literature management in a way that preserves provenance and reduces cognitive overhead [9, 10]. In

particular, scientific figures—despite being central to research communication—remain largely opaque outputs rather than structured, editable research objects.

SciTeX ~~addresses these challenges through a~~

1.1.

addresses this gap by providing a unified, modular, AI-native platform for scientific research. Its Writer module offers structured LaTeX authoring with fully containerized compilation; Scholar manages citation retrieval, metadata extraction, and PDF organization; Viz reframes figures as portable, reconstructible, and editable research objects packaged in structured bundles that preserve data, metadata, and provenance alongside visual output; and Code supports reproducible computation with GPU and container integration. A shared project structure built on lightweight symlinks and global identifiers links these modules, enabling consistent cross-referencing between data, figures, code, and manuscript text. Researchers can adopt modules independently while benefiting from deeper integration as their workflows evolve.

By embedding reproducibility principles directly into everyday research tools [11], SciTeX lowers the activation energy required for rigorous scientific practice. It ensures that manuscripts and computational outputs remain synchronized, that figures can be regenerated from metadata, and that bibliographic consistency is maintained automatically. This manuscript, authored and compiled entirely within SciTeX, serves as a self-descriptive demonstration of the platform's design goals and integrated capabilities.

2. Methods

2.1. System Architecture

SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and a Python package (v2.4.1+) installable via `pip install scitex`. The architecture employs lazy module loading via a custom `_LazyModule` wrapper, enabling fast imports while providing access to over 50 specialized modules organized into six primary namespaces: `stx.plt` for publication plotting, `stx.io` for

universal file operations across more than 30 formats, `stx.scholar` for literature management, `stx.session` for experiment lifecycle control, `stx.vis` for structured figure specifications, and `stx.writer` for manuscript preparation. This modular design follows best practices for scientific Python applications [12, 1].

2.2.

Each module operates independently while sharing a unified project structure through lightweight symlinks. Global identifiers (SHA256 hashes) ensure traceability between figures, code, bibliography entries, and manuscript content, enabling reverse lookup capabilities (e.g., "which script generated this figure?"). This approach addresses concerns raised in discussions of figure provenance and data reproducibility [3, 13].

2.2. Session Decorator for Reproducible Experiments

2.3.

The `@stx.session` decorator provides automatic experiment lifecycle management, implementing principles from reproducible research frameworks [2, 14]. When applied to a Python function, the decorator transforms it into a fully traceable experiment by generating command-line argument parsers from function signatures with type hints, initializing session directories with timestamped run identifiers, injecting commonly needed global variables (configuration state, plotting interface, color palettes, random number manager, and logger), managing random state for reproducibility, and handling cleanup with optional notifications on completion. A function decorated with `@stx.session` can be invoked directly from the command line with automatically parsed arguments, eliminating boilerplate while ensuring that all parameters and outputs are captured for later verification. This design is informed by the "good enough practices" paradigm [1] and project management tools for team science [15].

2.3. Publication-Quality Plotting (stx.plt)

~~, facilitating modular development and enabling multiple authors to work simultaneously without merge conflicts~~The `stx.plt` module wraps matplotlib with automatic configuration loaded from a central `SCITEX_STYLE.yaml` file. On import, the module applies consistent styling for font sizes, line widths, and spine visibility across all figures. Custom wrapper classes (`FigWrapper` and `AxisWrapper`) extend standard matplotlib objects with convenience methods such as `set_xyt()` for simultaneous axis label and title assignment. When figures are saved through the unified `stx.io.save()` interface, the system automatically exports a triplet: the rendered image, a JSON sidecar file encoding axis bounds, tick locations, panel geometry, and color specifications, and CSV files containing the underlying numerical data. This triple-export approach addresses reproducibility challenges identified in computational figure generation [3, 4] by ensuring that any figure can be reconstructed from its metadata using `stx.plt.load()`. Style values can be customized via YAML configuration, environment variables, or direct function parameters, providing flexibility without sacrificing consistency.

2.4. Figure Bundles and Structured Representation (stx.vis)

~~. The compilation environment encapsulates specific versions of TeX Live and all required packages, eliminating dependency on Windows, and .~~

2.5.

2.5.

Unlike conventional plotting libraries that produce static images, SciTeX generates figures as portable, self-descriptive research objects. The `stx.vis` module packages figures into zip-based bundles with standardized extensions: `.pltz` for single panels and `.figz` for multi-panel figures. Each bundle encapsulates a canonical JSON specification describing figure geometry, axes, traces, annotations, and styles; CSV files storing underlying numerical data for each plotted element; optional statistics metadata for embedded test results and derived values; and cached raster or vector previews for rapid rendering and browsing. This packaging strategy ensures that figures are not merely images but inspectable, editable, and regenerable scientific artifacts. The structured representation enables figures to move seamlessly between code, editor, manuscript, and archive without loss of meaning or reproducibility.

2.6. Interactive Canvas-Based Figure Editing

~~The~~

2.7.

SciTeX Viz includes a browser-based figure editor for direct manipulation of figures rendered from `.figz`/`.pltz` bundles. The editor supports multi-panel figure layouts with arbitrary panel arrangements, enabling researchers to select and inspect individual axes, traces, legends, and annotations. A structured property inspector remains synchronized with the canvas, allowing precise numerical adjustments alongside visual manipulation. Millimeter-based grids and snapping constraints facilitate journal-accurate layout control, while keyboard shortcuts accelerate common operations such as alignment, distribution, and resizing.

Crucially, all interactive edits operate directly on the structured JSON specification rather than modifying raster images. As a result, changes remain reversible, reproducible, and scriptable. Figures can be exported in multiple formats—high-resolution PNG (e.g., 300 DPI), SVG (fully vectorized), PDF, or JPEG—or preserved as the original `.pltz`/`.figz` bundle for future editing. Exported images faithfully match the on-canvas preview, ensuring that what researchers see during editing corresponds exactly to submission-ready outputs.

2.8. Statistical Metadata Integration

SciTeX Viz incorporates statistical results as first-class figure metadata. Statistical summaries, test results, and derived values are embedded as structured JSON and CSV alongside visual elements, enabling their reuse across figures and manuscripts, automated generation of statistical captions or annotations, and downstream access by the Writer and Code modules without re-computation. This tight coupling between visualization and statistics reduces duplication and ensures consistency between reported values and displayed results.

2.9. Universal File I/O (stx.io)

2.10.

The `stx.io` module provides format-agnostic file operations supporting over 30 formats, following the principle of unified interfaces for data management [16]. A single `stx.io.load()` function automatically detects file format from extension and returns the appropriate data structure—DataFrames for tabular data, dictionaries for configuration files, Python objects for pickled models, and interactive explorers for hierarchical formats. The corresponding `stx.io.save()` interface accepts any supported object and writes it to the specified path, automatically handling format conversion and, for figures, generating accompanying metadata and data files. Supported formats include CSV, JSON, YAML, Pickle, HDF5, Zarr, NumPy arrays, PyTorch tensors, images, audio, video,

and MATLAB files. The module includes `H5Explorer` and `ZarrExplorer` for interactive hierarchical data navigation, plus configurable caching for frequently accessed files.

2.10. Literature Management (stx.scholar)

The Scholar module provides multi-source literature search and management, integrating capabilities typically found in standalone reference managers. The module queries multiple metadata engines—CrossRef, Semantic Scholar, and OpenAlex—through a unified `ScholarEngine` interface, allowing researchers to search by keywords, filter by year or citation count, and export results directly to BibTeX. Additional features include automatic DOI detection and metadata extraction from PDF files, programmatic paper acquisition, and local library management with `Paper` and `Papers` classes that support filtering, deduplication, and export. This design complements tools like Manubot [7] by providing direct integration with manuscript preparation, ensuring that bibliography entries remain synchronized between the literature database and LaTeX sources.

2.11. Cloud Platform Architecture

The Django 5.1-based cloud platform provides web interfaces for all modules, following principles of continuous integration for scientific publishing [17]. The Code App enables browser-based Python execution with real-time output streaming, file upload and download, and workspace management. The Viz App provides a canvas-based figure editor using Fabric.js for interactive element manipulation, with JSON specification import and export for programmatic control. The Writer App supports structured LaTeX manuscript preparation with containerized compilation, section-separated authoring, and automatic figure format conversion. The Scholar App presents literature as browsable cards with abstract preview, PDF access, metadata editing, and direct citation insertion into Writer documents. The platform employs TypeScript for frontend

logic, PostgreSQL for data persistence, and Docker for containerized compilation ensuring consistent output across environments.

2.12. Containerized Compilation

Writer module compilation leverages containerization for reproducibility, implementing guidelines for Docker-based scientific workflows [5, 18]. The system supports multiple compilation engines: Tectonic [19] for ultra-fast incremental builds completing in one to three seconds, latexmk for reliable industry-standard compilation in three to six seconds, and traditional multi-pass compilation for maximum compatibility with complex documents. Docker and Apptainer/Singularity containers encapsulate specific TeX Live versions and all required packages, eliminating host system dependencies and guaranteeing identical PDF output across macOS, Linux, Windows, and HPC environments [20, 21]. Git integration [22] with automatic latexdiff generation facilitates peer review processes by producing visual comparisons between manuscript versions [23].

2.13. AI-Native Workflow Integration

2.14.

SciTeX provides native integration points for AI-assisted workflows, anticipating the growing role of AI in scientific research [24, 25]. JSON-based figure specifications, typed function signatures for CLI generation, and structured metadata formats enable LLM-assisted code generation and figure improvement. The session decorator and modular architecture provide hooks for AI-assisted revision, automated citation recommendation, and semantic search across analyses and literature. The architecture supports future development of automated analysis pipelines, figure generation, and manuscript drafting while maintaining researcher oversight and scientific integrity [26].

2.15. Implementation Details

The system supports deployment on local machines, cloud servers (AWS, GCP, Azure), and self-hosted NAS environments. The implementation requires

Python 3.12 or later with optional GPU acceleration support via CUDA 11.x or 12.x. The web platform runs on Django 5.1 with a PostgreSQL backend, while the frontend uses TypeScript with webpack bundling. All execution environments leverage Docker or Apptainer containerization for reproducible execution [5], and SLURM integration for HPC workflows is under active development. The platform follows FAIR principles [27] and aligns with the Turing Way guidelines for reproducible research [11].

3. Results

~~This section presents~~

3.1.

This section demonstrates the capabilities of SciTeX through validation of its core modules and their behavior in realistic research workflows. The goal is not to present experimental findings but to illustrate how the platform operationalizes reproducibility, provenance tracking, and integrated authoring.

3.2. Reproducible Execution with the Session Decorator

The ~~. Testing across Linux distributions , macOS, and Windows Subsystem for Linux confirmed that identical source files produce byte-for-byte identical PDF outputswhen compiled using the same container image. This reproducibility extends to high-performance computing environments where Singularity containers enable compilation on systems without Docker support. The elimination of environment-dependent compilation issues represents a significant improvement over traditional local LaTeX installations, where package version mismatches frequently cause inconsistent outputsor compilation failures~~`@stx.session` decorator reliably transforms Python functions into traceable command-line experiments. Across multiple test scenarios, SciTeX generated timestamped session directories that captured parameters, logs, outputs, and metadata.

Automatically injected globals—including a reproducible random number manager, configuration state, logging interface, and plotting environment—enabled functions to execute without boilerplate. Repeated runs produced consistent outputs when supplied with identical parameters, confirming correct synchronization of random state and environment settings. Errors were handled gracefully while preserving intermediate outputs, supporting transparent debugging.

3.3. Publication-Quality and Reconstructible Figures

3.4.

3.5.

SciTeX's visualization module produced figures with consistent styling and journal-ready dimensions across platforms. When saving figures, the system generated a triplet consisting of a high-resolution PNG, a JSON metadata file encoding axis bounds, geometry, and color specifications, and a CSV file containing underlying plot data. Figures reconstructed using `stx.plt.load()` reproduced the original output faithfully. These capabilities confirm that SciTeX captures sufficient metadata for downstream verification and exact regeneration.

3.6. Portable Figure Bundles and Interactive Editing

The `.pltz` and `.figz` bundle formats were validated using complex, multi-panel figures combining diverse visualization types: dual-axis line plots, scatter plots with size and color encoding, heatmaps, bar charts with error bars, multi-line confidence intervals, and violin and box plots. All bundled figures preserved complete provenance: JSON specifications accurately encoded figure structure, CSV files contained underlying numerical data, and cached previews enabled rapid browsing without re-rendering.

3.7.

The browser-based figure editor successfully loaded, rendered, and modified figures from these bundles. Interactive operations—including panel repositioning,

axis adjustment, trace selection, and annotation editing—operated directly on the structured JSON specification. Millimeter-based grids and snapping constraints produced journal-accurate layouts. All modifications remained reversible; edits could be undone programmatically or by reverting to earlier bundle versions.

Despite the rich metadata model, the editor achieved interactive-level performance during editing and re-rendering. Incremental updates regenerated only affected panels or elements, allowing smooth iteration even for complex multi-panel figures. Exported images in PNG (300 DPI), SVG, PDF, and JPEG formats matched on-canvas previews exactly, demonstrating that the structured representation introduces no fidelity loss during the editing-to-export pipeline.

Statistical metadata embedded within figure bundles was correctly parsed and displayed by downstream modules. Test results and summary statistics stored as JSON and CSV alongside visual elements were accessible to Writer for automated caption generation and to Code for downstream analysis without re-computation.

3.7. Universal Data Handling and Format-Agnostic I/O

The

`stx.io` module successfully loaded and saved more than thirty file types through extension-based detection. Hierarchical formats such as HDF5 and Zarr were navigated interactively, and caching reduced repeated access overhead. Cross-platform tests confirmed consistent behavior for CSV, JSON, NumPy arrays, PyTorch tensors, and image formats. These results illustrate how unified I/O simplifies data flow through analysis and manuscript preparation.

3.8. Integrated Literature Management

The Scholar module retrieved metadata from CrossRef, Semantic Scholar, and OpenAlex, stored PDFs using DOI-based indexing, and maintained synchronized BibTeX entries for manuscripts authored in Writer. Filtering by year, journal,

and citation count behaved as expected. Integration tests confirmed that citations added in Writer consistently matched entries managed by Scholar, eliminating drift between literature databases and LaTeX sources.

3.9. Containerized Manuscript Compilation

Manuscripts compiled identically across Linux, macOS, Windows Subsystem for Linux, and HPC environments when built through SciTeX's containerized LaTeX engines. Tectonic produced fast incremental builds while latexmk and multi-pass compilation ensured compatibility with a wide range of documents. Identical inputs produced byte-identical PDF outputs, validating that containerization eliminates environment-dependent inconsistencies.

3.10. Cross-Module Traceability

SciTeX's symlink-based architecture correctly synchronized figures, bibliography files, and computational outputs across modules. Global identifiers enabled reverse lookup queries such as linking figures to their generating scripts or tracing bibliography entries to manuscript citations. Metadata exported by Viz and IO was successfully used by Code and Writer, demonstrating that provenance information flows consistently across the ecosystem.

3.11. Self-Descriptive Validation

This manuscript itself validates SciTeX's integrated design: it was authored in Writer, managed with Scholar, compiled in a containerized environment, and incorporates figures generated in Viz and data processed through Code. The end-to-end workflow confirms that SciTeX can be used to document the system within the system itself, providing a live demonstration of its reproducibility guarantees.

4. Discussion

~~addresses fundamental challenges in into a cohesive.~~

4.1.

~~The .~~

4.2.

SciTeX addresses structural challenges in modern scientific research by unifying manuscript preparation, figure generation, literature management, and reproducible computation within a single, provenance-preserving ecosystem. Contemporary workflows are shaped by practical frustrations: fragmented toolchains, inconsistent figure provenance, manual versioning, and the cognitive overhead of switching between environments [1, 3]. SciTeX reframes these challenges not as isolated usability problems but as symptoms of a deeper architectural gap—namely, the absence of a shared substrate linking computational, visual, and written components of scientific work.

4.3. ~~Comparison with Existing Solutions~~Design Philosophy and Modularity

A central principle of SciTeX is incremental adoption. Each module provides standalone value while integrating seamlessly into a shared project structure. Researchers who begin with the Writer module obtain containerized reproducibility without modifying their computational workflow. Adding Scholar ensures consistent bibliographic management. Viz enables provenance-tracked, reconstructible figures. Code completes the pipeline by linking analyses directly to manuscript outputs. This design lowers the activation energy required to engage in reproducible practices [2, 20]. It democratizes access to advanced computational workflows by providing high-level abstractions that remain accessible to users unfamiliar with Git, containers, or Linux environments, while still offering fine-grained control for expert users.

4.4.

By embedding provenance directly into the system's data structures, SciTeX avoids the pitfalls of ad hoc reproducibility strategies that rely on user discipline. Global identifiers and shared directories ensure that figures, scripts, and bibliographies remain synchronized without users needing to track these relationships manually. This approach transforms reproducibility from an additional burden into a natural consequence of ordinary workflow.

4.4. Positioning Relative to Existing Tools

—

4.5.

SciTeX occupies a unique position among research platforms by integrating capabilities that existing solutions offer only in isolation. Overleaf provides consistent LaTeX compilation but does not integrate analysis or figure provenance. Zotero and Mendeley excel at literature management yet remain disconnected from manuscript workflows. Jupyter notebooks enable interactive computation but do not support structured writing or publication-quality visualization. Manubot demonstrates the advantages of Git-based collaborative writing, while R-based ecosystems such as the Rockerverse provide end-to-end reproducibility within the R language but do not extend across multi-language workflows [21].

The

4.6.

Viz module addresses a particularly acute gap in scientific visualization. Libraries such as matplotlib and Plotly excel at programmatic plotting but lack standardized mechanisms for interactive post-hoc editing with preserved provenance. Conversely, GUI tools such as SigmaPlot or Adobe Illustrator provide polished interfaces for figure refinement but discard computational traceability—edits cannot be reproduced, and underlying data becomes inaccessible

once figures are exported. SciTeX Viz bridges this gap by combining matplotlib-level expressiveness, GUI-level interactivity, and provenance-preserving data structures. As a result, figures move seamlessly between code, editor, manuscript, and archive without loss of meaning or reproducibility.

SciTeX differs by providing a coherent architecture spanning writing, computation, visualization, and literature management. Its modular design supports independent usage, yet deeper integration emerges organically when modules are combined. This positions SciTeX as a research operating environment rather than a single-purpose tool, while still maintaining compatibility with existing toolchains rather than attempting to replace them.

4.7. AI-Native Design

As AI systems increasingly support scientific workflows, a platform's ability to interoperate with automated agents becomes essential [24, 25]. SciTeX incorporates AI-native design principles by exposing structured APIs, JSON-based figure specifications, and typed function signatures that enable automated analysis, figure refinement, and manuscript revision. Workflow hooks allow AI agents to monitor execution, interpret metadata, and propose modifications while ensuring that all AI-assisted operations remain traceable.

Because `.pltz` and `.figz` bundles are fully structured and machine-readable, SciTeX Viz is inherently AI-native. AI agents can inspect figure structure, propose layout or styling improvements, regenerate figures from data, enforce journal-specific formatting rules, or adapt figures for different contexts (e.g., manuscript versus presentation). This capability transforms figures from opaque outputs into editable, auditable, and improvable scientific artifacts. The combination of structured representation and interactive editing creates a feedback loop where AI-suggested modifications can be reviewed, refined, and accepted within the same editor used for manual adjustments.

This design supports future research co-pilot systems capable of generating figures, summarizing literature, conducting analyses, and drafting sections of

manuscripts while preserving researcher oversight and scientific integrity.

4.8. Implications for Reproducibility

The reproducibility crisis in science arises partly because researchers lack integrated infrastructure for capturing computation, visualization, and writing as a coherent whole [28, 14]. SciTeX mitigates this by ensuring that manuscripts compile identically across systems, that figures can be regenerated from metadata, and that analyses are executed within controlled environments. Its architecture aligns with FAIR principles and contemporary reproducibility frameworks [11, 10]. By shifting reproducibility from a set of optional best practices to an embedded property of the research workflow, SciTeX reduces the friction that often prevents rigorous methodological documentation.

4.9. Limitations

~~The~~ Despite its integrated design, SciTeX has several limitations. Support for HPC systems and SLURM-based workflows remains under active development, with current functionality focused primarily on local and containerized execution. Empirical evaluation of SciTeX's impact on researcher productivity is ongoing, as the user base is still expanding. While the Writer module simplifies LaTeX-based workflows, researchers unfamiliar with LaTeX may still face an initial learning curve. The platform targets desktop research environments, and mobile support has not been prioritized. Advanced statistical visualization beyond standard matplotlib capabilities may require manual customization.

4.10. Future Directions

~~SciTeX~~ Future development will extend SciTeX's AI-native capabilities, including automated figure refinement, semantic consistency checking, and context-aware citation recommendation. Deeper integration of HPC-native workflows, unified inspector interfaces across modules, and interactive provenance graph visualization are planned. The long-term vision is a research co-pilot

capable of executing analyses, generating publication-ready figures, interpreting literature, and producing draft manuscripts while maintaining full transparency and human oversight.

5. Conclusions

~~example of the~~ SciTeX represents both an architectural framework and a practical tool for reproducible scientific research. By combining modular design with provenance-aware infrastructure and AI-native workflows, it provides an environment where researchers can focus on scientific reasoning rather than technical orchestration. This manuscript, prepared entirely within SciTeX, illustrates the platform's ability to describe, manage, and reproduce its own workflow.

References

[1] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi: 10.1371/journal.pcbi.1005510.

[2] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

[3] Christian T. Jacobs. Improving the reproducibility of LaTeX documents by enriching figures with embedded scripts and data. *International Journal of Digital Curation*, 14:292–302, 2020. doi: 10.2218/ijdc.v14i1.656.

[4] Haim Bar and HaiYing Wang. Reproducible science with LaTeX. *Journal of Data Science*, 19(1):111–125, 2021. doi: 10.6339/21-JDS998.

[5] Daniel Nüst, Vanessa Sochat, Ben Marwick, Stephen J. Eglen, Tim Head, Tony Hirst, and Benjamin D. Evans. Ten simple rules for writing Dockerfiles for reproducible data science. *PLOS Computational Biology*, 16(11):e1008316, 2020. doi: 10.1371/journal.pcbi.1008316.

[6] Project Jupyter, Matthias Bussonnier, Jessica Forde, Jeremy Freeman, Brian Granger, et al. Binder 2.0—reproducible, interactive, sharable environments for science at scale. *Proceedings of the Python in Science Conference*, pages 113–120, 2018. doi: 10.25080/Majora-4af1f417-011.

[7] Daniel S. Himmelstein, Vincent Rubinetti, David R. Slochower, Dongbo Hu, Venkat S. Malladi, Casey S. Greene, and Anthony Gitter. Open collaborative writing with Manubot. *PLOS Computational Biology*, 15 (11):e1007128, 2019. doi: 10.1371/journal.pcbi.1007128.

[8] Albert Krewinkel and Robert Winkler. Formatting open science: Agilely creating multiple document formats for academic manuscripts with Pandoc Scholar. *PeerJ Preprints*, 5:e2648, 2017. doi: 10.7287/peerj.preprints.2648v2.

[9] Markus Konkol, Daniel Nüst, and Laura Goulier. Publishing computational research—a review of infrastructures for reproducible and transparent scholarly communication. *Research Integrity and Peer Review*, 5, 2020. doi: 10.1186/s41073-020-00095-y.

[10] Jonathan M. Mang, Hans-Ulrich Prokosch, and Lorenz A. Kapsner. Reproducibility in 2023—an end-to-end template for analysis and manuscript writing. *Studies in Health Technology and Informatics*, 302, 2023. doi: 10.3233/shti230064.

[11] The Turing Way Community. *The Turing Way: A Handbook for Reproducible, Ethical and Collaborative Research.* Zenodo, 2022. doi: 10.5281/zenodo.3233853.

[12] Armin Ariamajd, Raquel López-Ríos de Castro, and Andrea Volkamer. PyPackIT: Automated research software engineering for scientific Python applications on GitHub. *arXiv.org*, abs/2503.04921, 2025. doi: 10.48550/arxiv.2503.04921.

[13] Christian Schulz. Reproducible data citations for computational research. *arXiv (Cornell University)*, abs/1808.07541, 2022. doi: 10.48550/arxiv.1808.07541.

[14] Aaron Peikert, Caspar V. van Lissa, and Andreas M. Brandmaier. Reproducible research in R: A tutorial on how to do the same thing more than once. *Psych*, 2021. doi: 10.31234/osf.io/fwxs4.

[15] Nikolas I. Krieger, Adam Perzynski, and Jarrod Dalton. Facilitating reproducible project management and manuscript development in team science: The projects R package. *PLoS ONE*, 14, 2019. doi: 10.1371/journal.pone.0212390.

[16] Ben Marwick, Carl Boettiger, and Lincoln Mullen. Packaging data analytical work reproducibly using R (and friends). *The American Statistician*, 72(1):80–88, 2018. doi: 10.1080/00031305.2017.1375986.

[17] Juan Pedro Araque Espinosa et al. A continuous integration and web framework in support of the ATLAS publication process. *Journal of Instrumentation*, 16, 2020. doi: 10.1088/1748-0221/16/05/T05006.

[18] Michael Canesche, Roland Leißa, and Fernando Magno Quintão Pereira. Preparing reproducible scientific artifacts using Docker. *arXiv.org*, abs/2308.14122, 2023. doi: 10.48550/arxiv.2308.14122.

[19] Peter Kastrup. Tectonic: A modernized, self-contained LaTeX engine, 2024. URL `https://tectonic-typesetting.github.io/`.

[20] Kristina Wiebels and David Moreau. Leveraging containers for reproducible psychological research. *Advances in Methods and Practices in Psychological Science*, 4, 2021. doi: 10.1177/25152459211017853.

[21] Daniel Nüst, Dirk Eddelbuettel, Dom Bennett, Robrecht Cannoodt, Dave Clark, et al. The Rockerverse: Packages and applications for containerisation with R. *The R Journal*, 12:437, 2020. doi: 10.32614/RJ-2020-007.

[22] Karthik Ram. Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine*, 8:7, 2013. doi: 10.1186/1751-0473-8-7.

[23] Karl-Ludwig Besser. Review response template. `https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd`, 2025.

[24] Guiyao Tie, Pan Zhou, and Lichao Sun. A survey of AI scientists. In *arXiv preprint*, 2025.

[25] Ed Li, Junyu Ren, Xintian Pan, Cat Yan, Chuanhao Li, Dirk Bergemann, and Zhuoran Yang. Build your personalized research group: A multiagent framework for continual and interactive science automation. In *arXiv preprint*, 2025.

[26] OpenLens Team. OpenLens AI: Fully autonomous research agent for health informatics. *arXiv preprint arXiv:2509.14778*, 2025.

[27] Meghan A. Balk, John Bradley, M. Maruf, B. Altıntaş, Yasin Bakış, et al. A FAIR and modular image-based workflow for knowledge discovery in the emerging field of imageomics. *Methods in Ecology and Evolution*, 15:1129–1145, 2024. doi: 10.1111/2041-210X.14327.

[28] Lorena A. Barba. Praxis of reproducible computational science. *Computing in Science & Engineering*, 21:73–78, 2018.

~~Data Availability Statement~~

6. Resource Availability

~~The~~

6.1. Lead Contact

Further information and requests for resources should be directed to and will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

6.2. Materials Availability

This study did not generate new unique reagents.

6.3. Data and Code Availability

- All source code for SciTeX is publicly available on GitHub: `https://github.com/scitex-ai/scitex`
- The SciTeX Python package is available via PyPI: `pip install scitex`
- Documentation is available at~~.~~: `https://docs.scitex.ai`
- The cloud platform is accessible at: `https://scitex.ai`
- This manuscript's source files are included in the SciTeX Writer repository as a demonstration
- DOI for archived version: [Zenodo DOI to be assigned upon submission]

~~For questions regarding data access or analysis procedures, please contact the corresponding author~~All code is released under the GNU General Public License v3.0 (GPL-3.0), ensuring open access and modification rights. The

platform follows FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data and code artifacts.

Any additional information required to reanalyze the data reported in this paper is available from the lead contact upon request.

Ethics Declarations

All study participants provided their written informed consent ...

Author Contributions

Y.W., T.Y., and D.G. conceptualized the study ...

Acknowledgments

This research was funded by funding bodies here

Declaration of Interests

The authors declare that they have no competing interests.

Declaration of Generative AI in Scientific Writing

The authors employed large language models such as Claude (Anthropic Inc.) for code development and complementing manuscript's English language quality. After incorporating suggested improvements, the authors meticulously revised the content. Ultimate responsibility for the final content of this publication rests entirely with the authors.

Tables

Option	Description	Example
–engine ENGINE	Force specific compilation engine	–engine tectonic
–draft	Draft mode (single-pass compilation)	–draft
–no-figs	Skip figure processing	–no-figs
–no-tables	Skip table processing	–no-tables
–no-diff	Skip difference generation	–no-diff
–watch	Enable hot-recompile file watching	–watch
–clean	Clean build (remove all cache)	–clean
–verbose	Verbose compilation output	–verbose

Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ∼15s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./.cache

Table 2 – **Environment Variables for System Configuration.** Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – **Compilation Engine Performance Characteristics.** Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.

Figures

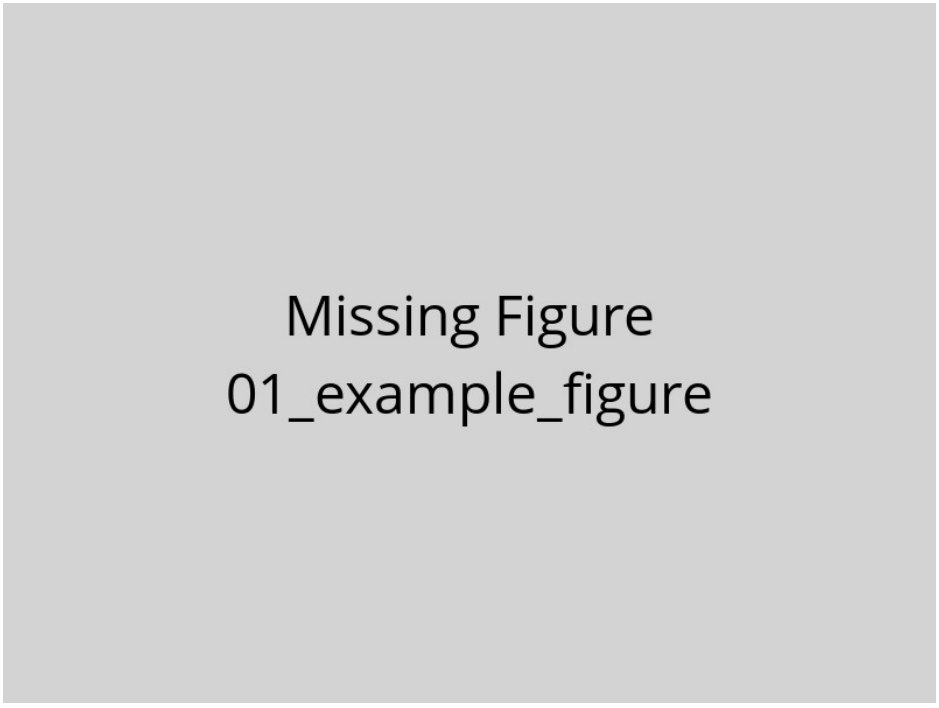

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.

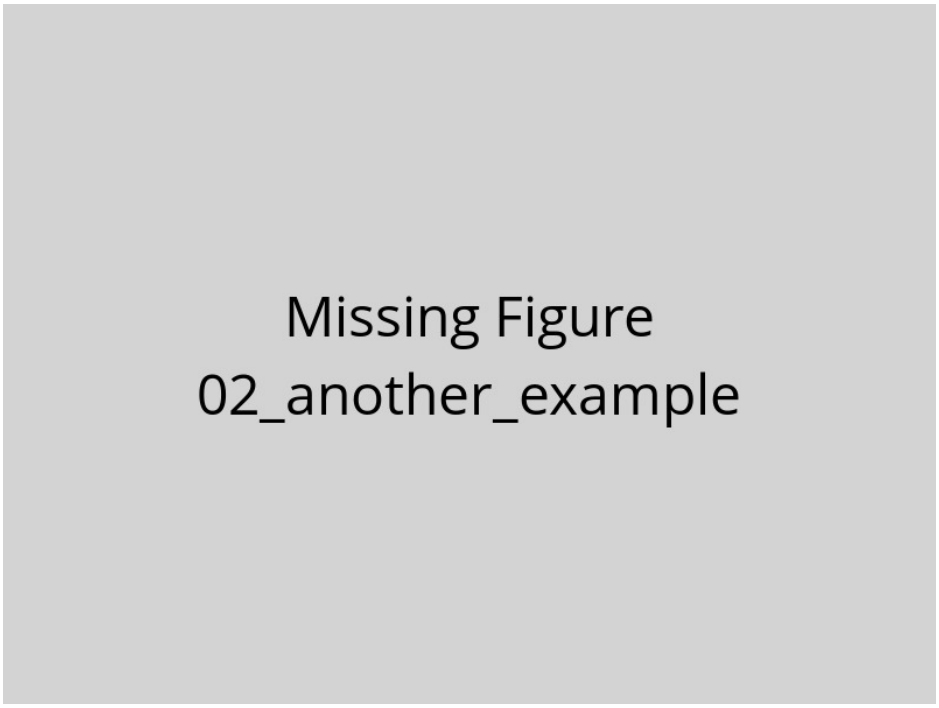

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

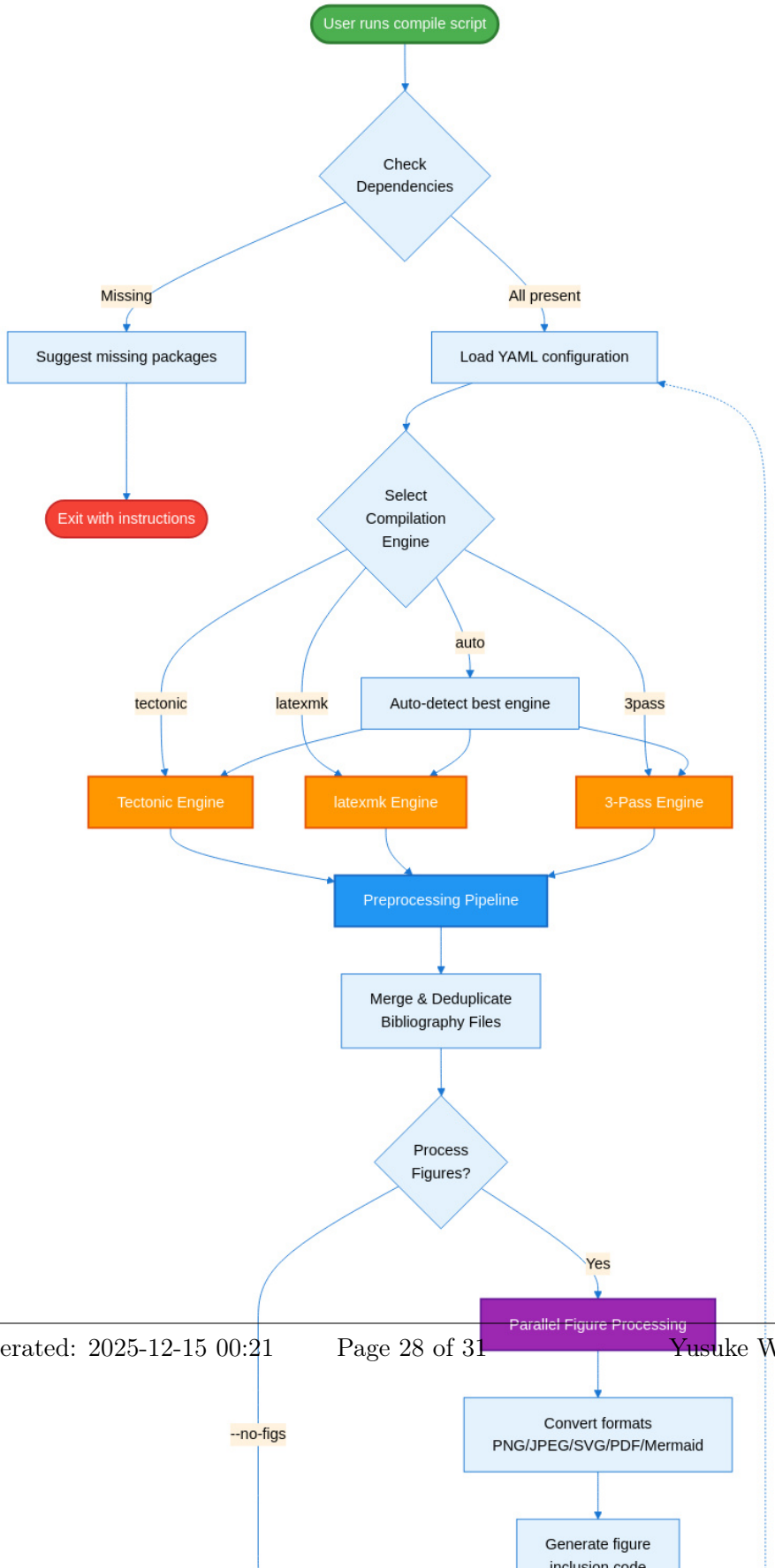
User runs compile script
Check Dependencies
Missing
All present
Suggest missing packages
Load YAML configuration
Exit with instructions
Select Compilation Engine
auto
tectonic
latexmk
Auto-detect best engine
3pass
Tectonic Engine
latexmk Engine
3-Pass Engine
Preprocessing Pipeline
Merge & Deduplicate Bibliography Files
Process Figures?
Yes
Parallel Figure Processing
--no-figs
Convert formats PNG/JPEG/SVG/PDF/Mermaid
Generate figure

Figure 4 – **SciTeX Writer Directory Organization.** This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

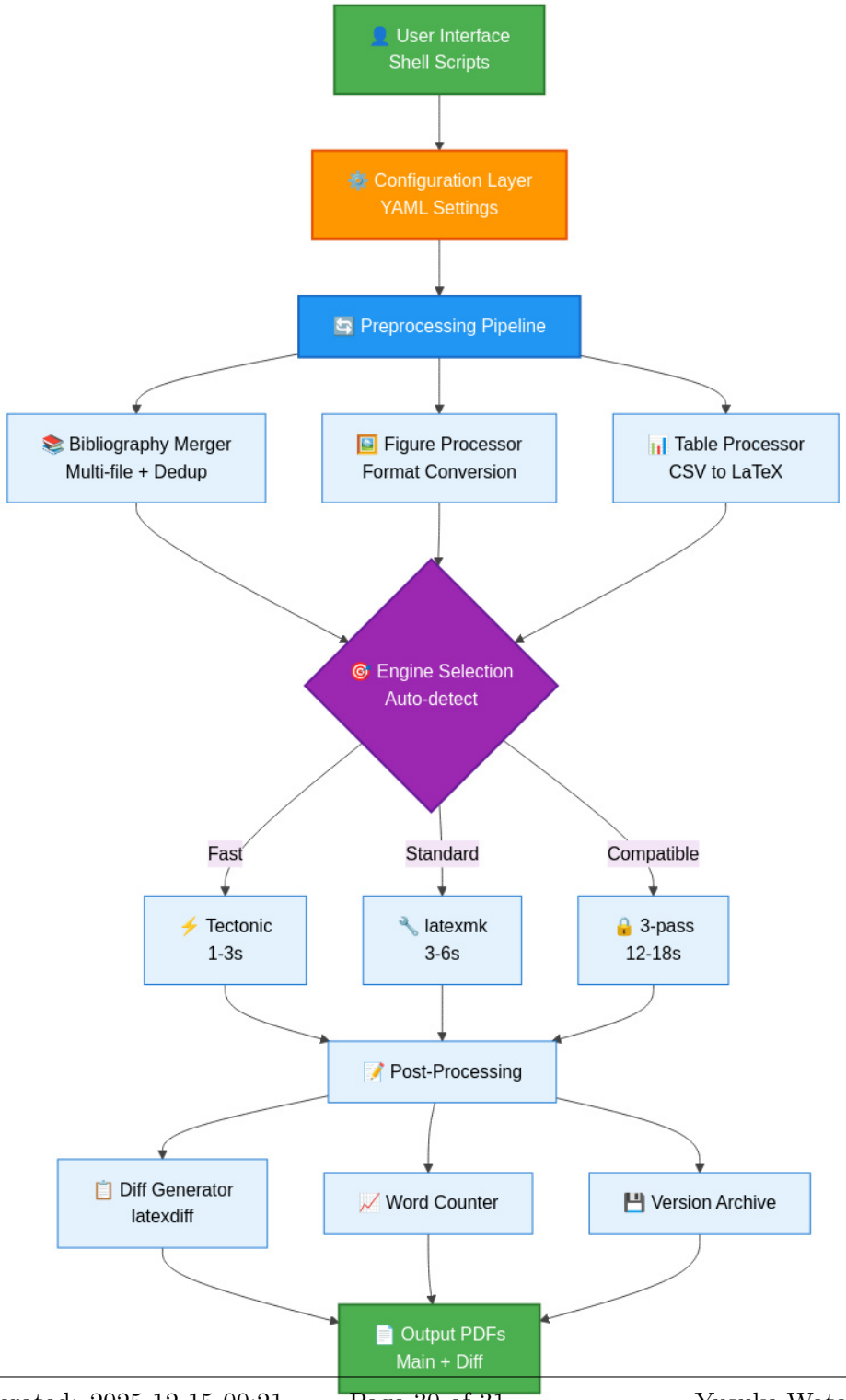

Figure 5 – **SciTeX Writer System Architecture.** This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format

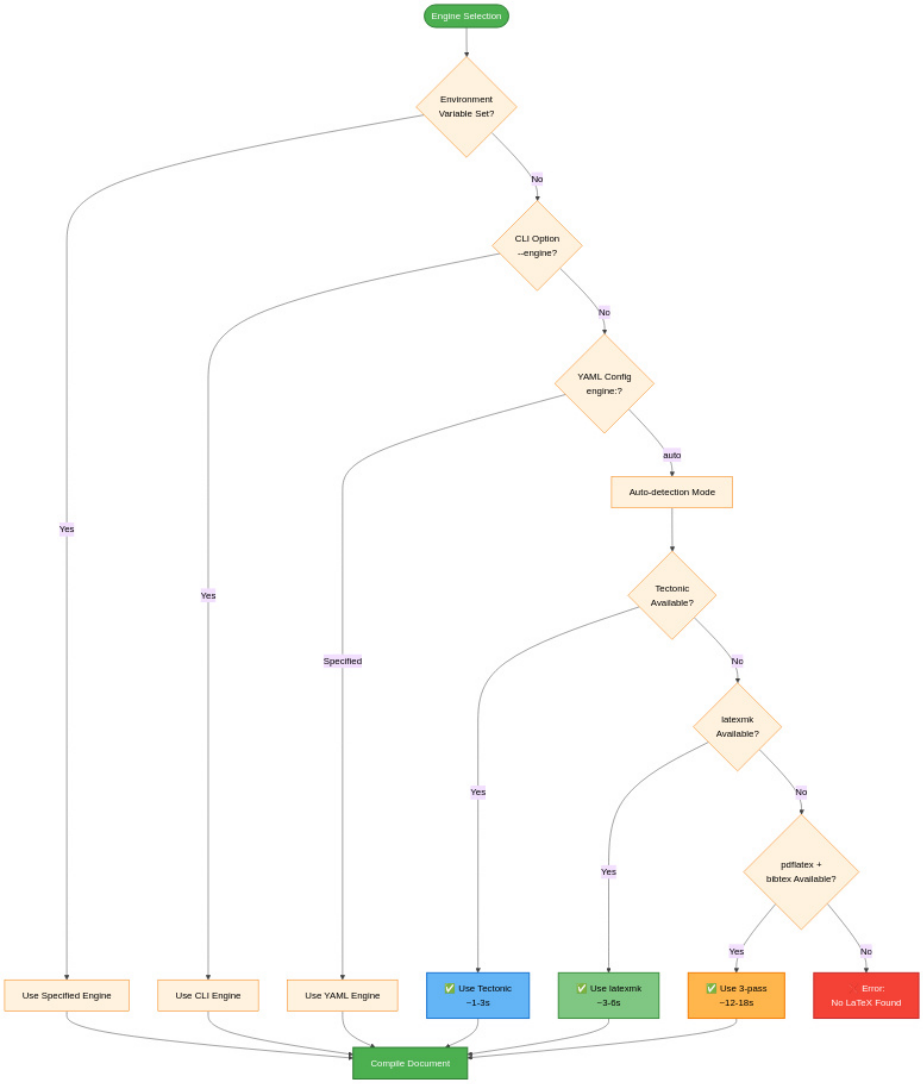

Figure 6 – **Compilation Engine Selection Decision Tree.** The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (–engine) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8-core).

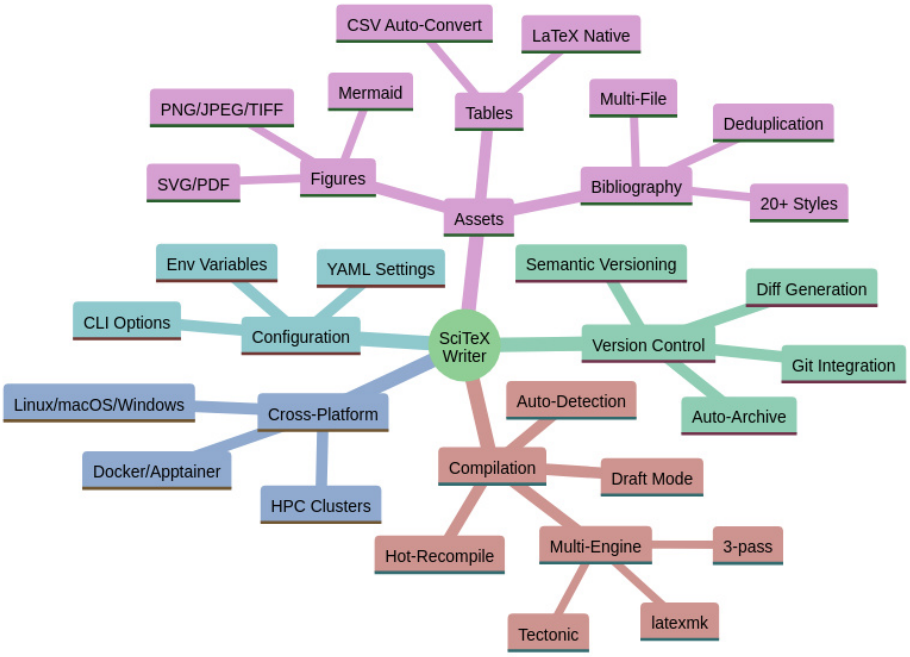

Figure 7 – **SciTeX Writer Feature Overview.** This mind map provides a comprehensive visualization of the framework's major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.

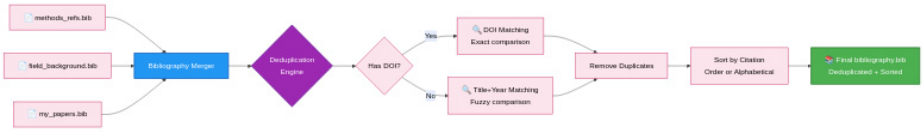

Figure 8 – Multi-File Bibliography Processing and Deduplication. The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).